Now on to the code. In the Catalog Service project, create a new `SecurityConfig` class in the `com.polarbookshop.catalogservice.config` package. Similar to what we did for Edge Service, we can use the DSL provided by `HttpSecurity` to build a `SecurityFilterChain` configured with the required security policies.

Listing 12.8 Configuring security policies and JWT authentication

```
@EnableWebSecurity                                          ❶
public class SecurityConfig {

  @Bean
  SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
      .authorizeHttpRequests(authorize -> authorize
        .mvcMatchers(HttpMethod.GET, "/", "/books/**")
          .permitAll()                                      ❷
        .anyRequest().authenticated()                       ❸
      )
      .oauth2ResourceServer(
       OAuth2ResourceServerConfigurer::jwt                  ❹
      )
      .sessionManagement(sessionManagement ->               ❺
        sessionManagement
          .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
      .csrf(AbstractHttpConfigurer::disable)                ❻
      .build();
  }
}
```

❶ Enables Spring MVC support for Spring Security

❷ Allows users to fetch greetings and books without being authenticated

❸ Any other request requires authentication.

❹ Enables OAuth2 Resource Server support using the default configuration based on JWT (JWT authentication)

❺ Each request must include an Access Token, so there's no need to keep a user session alive between requests. We want it to be stateless.

❻ Since the authentication strategy is stateless and doesn't involve a browser-based client, we can safely disable the CSRF protection.